FUNCTIONAL SOFTWARE TEST PLAN

for

Encost Smart Graph Project

Version 1.0

Prepared by: Min Soe Htut
SoTech Engineer

SoTech

April 24, 2025

Contents

Revision History

Name	Date	Reason for Changes	Version
Min Soe Htut	29/03/25	Initial Draft	0.1
Min Soe Htut	05/04/25	Draft for Black Box REQ 1,2	0.2
Min Soe Htut	12/04/25	Draft for Black Box REQ 3 to 7	0.3
Min Soe Htut	19/04/25	Draft for Black Box REQ 1 to 7	0.4
Min Soe Htut	20/04/25	Draft for White Box	0.5
Min Soe Htut	23/04/25	Draft Mutation Testing	0.6
Min Soe Htut	24/04/25	Draft JUnit	0.7
Min Soe Htut	24/04/25	Finisize	1.0

1 Introduction/Purpose

1.1 Purpose

The purpose of this document is to outline a Functional Software Test Plan for the Encost Smart Graph Project (ESGP). It is based on the requirements and design specifications agreed upon in the Software Requirements Specification (SRS) and Software Design Specification (SDS) documents. This plan defines how the system's functional behaviour will be verified through structured black-box and white-box testing approaches.

The ESGP is a command-line application that allows users to load and visualise Encost Smart Homes data as a graph. This testing document provides a systematic approach for evaluating whether the software meets its specified requirements, and whether it behaves correctly across its key features, such as user login, dataset handling, statistical analysis, and graph visualization. It also includes mutation testing and branch coverage to ensure the robustness of implementation.

This document is intended to guide the test-driven development of ESGP and to serve as a reference for developers responsible for building an implementation that passes all defined tests.

1.2 Document Conventions

The following conventions and terminology are used throughout this document:

- **SRS** – Software Requirements Specification
- **SDS** – Software Design Specification
- **ESGP** – Encost Smart Graph Project
- **UI** – User Interface
- **REQ-x** – Functional requirement number x from the SRS and SDS (e.g., REQ-1, REQ-5)
- **TCxx** – Test Case number xx as defined in this Test Plan (e.g., TC17)
- **EP** – Equivalence Partitioning (black-box testing technique)
- **BVA** – Boundary Value Analysis (black-box testing technique)

1.3 Intended Audience and Reading Suggestions

This document is intended for all stakeholders responsible for the testing and quality assurance of the Encost Smart Graph Project (ESGP). It defines the testing approach and details test cases designed to verify that the system meets its requirements as specified in the Software Requirements Specification (SRS) and implemented as per the Software Design Specification (SDS).

- **Software Testers:** Should focus on Sections 4 and 5 to understand the test case structure and apply the defined black-box testing techniques during test execution. Responsible for validating the implementation against expected outputs.
- **Software Engineers:** Should use this document alongside the SRS and SDS to ensure all defined functional behaviours are testable and their implementation will pass the provided unit tests.
- **Project Managers:** May review Sections 3 and 6 to assess test coverage and ensure alignment between the test plan, software requirements, and project deliverables.

1.4 Project Scope

The Encost Smart Graph Project (ESGP) is a console-based software application developed to visualize smart home device data collected by Encost. The system allows users to explore the structure and connectivity of devices using graph-based representations. It supports two types of users: *Community Users* and *Encost-Verified Users*.

Community users can access a preloaded visualisation of the Encost Smart Homes Dataset. Encost-verified users have access to additional features, including uploading custom datasets, viewing summary statistics, and generating personalised graph visualisations.

This Functional Software Test Plan outlines a systematic black-box testing strategy designed to verify that ESGP satisfies its specified requirements, as defined in the Software Requirements Specification (SRS) and implemented in the Software Design Specification (SDS). Each high-priority functional requirement is tested to ensure correctness for both user types.

Out of Scope: Internal code-level (white-box) testing, performance/stress testing, GUI-based interface testing, and security validation are not included in this plan. These may be addressed in future development phases.

2 Specialized Requirements Specification

During the review of the Software Requirements Specification (SRS) and Software Design Specification (SDS), several clarifications were made following discussion with the client (course lecturers). These clarifications directly influence the scope and focus of this test plan.

- **Backups:** While developers are expected to maintain regular backups of their source code, the method and frequency are left to their discretion. Backup validation is considered outside the scope of this test plan.
- **Dataset Anonymisation:** All datasets used in ESGP are assumed to be anonymised prior to testing. No verification of data anonymisation is required or tested.
- **Software Distribution:** The ESGP application will be distributed as a local installation package. Encost will provide installation instructions to end-users. Installation process testing is not required.
- **Password Encryption (REQ-5):** Although the system is expected to handle password encryption internally, this behaviour is not directly testable through external black-box methods. Instead, login success/failure is tested to confirm the correctness of authentication and encryption match logic.

3 Black-box Testing

3.1 Requirement #1

User Categorisation (Community or Encost)

3.1.1 Description

At application start-up, ESGP prompts users to identify whether they are an Encost user. Their response determines their access level. Encost users proceed to login, while Community users are taken directly to a limited feature menu. This step is crucial for establishing the user's access rights and sets the path for different system behaviours.

3.1.2 Functional Requirements Tested

- **SRS 4.1.3 REQ-1:** A prompt should allow the user to indicate whether they are a community member or an Encost member.
- **SRS 4.1.3 REQ-2:** The system should store the selected user type (community or encost-unverified).
- **SRS 4.1.3 REQ-3:** Based on the response, the system should either show the ESGP Feature Options (Community) or direct the user to the ESGP Account Login screen (Encost).

3.1.3 Test Type

Test Level: System-level black-box test
Test Technique(s):

- **Equivalence Partitioning** – to test the system's interpretation of valid versus invalid user type inputs.
- **Boundary Value Analysis** – to check edge conditions such as empty input and irregular formatting.

3.1.4 Test Cases

Test ID	Input	Expected Output	Equivalence Class
TC01	"yes"	Redirect to Encost login screen	Valid input – Encost
TC02	"no"	Show ESGP Feature Options (limited access)	Valid input – Community
TC03	"YES", "Yes", "yEs"	Treated as "yes"; login screen shown	Case insensitivity (Valid)
TC04	"No", "nO"	Treated as "no"; community menu shown	Case insensitivity (Valid)
TC05	Empty input (" ")	Error message, user re-prompted	Invalid input – Empty
TC06	Invalid inputs ("maybe", "123", "!@*")	Error message, user re-prompted	Invalid input – Unexpected

Table 3.1: Black-box test cases for SRS 4.1.3 – Categorising Users using Equivalence Partitioning

Table 3.1 summarises the black-box tests that validate how ESGP categorises users at system launch. It ensures the system distinguishes valid input from unexpected or invalid entries and handles them appropriately.

3.2 Requirement #2

ESGP Account Login (Encost Users)

3.2.1 Description

This test covers the full login process for Encost users, starting from prompting for credentials to granting access upon successful authentication. It verifies both valid and invalid input scenarios, appropriate user feedback, and access control. The test also assumes that passwords are encrypted and validated securely against a stored dataset.

3.2.2 Functional Requirements Tested

- **SRS 4.2.3 REQ-1:** The ESGP Account Login prompt should first prompt the user to enter their username. It should then prompt the user to enter their password.

- **SRS 4.2.3 REQ-2:** Once the username and password have been entered, the system should check that the inputs are valid.
- **SRS 4.2.3 REQ-3:** If the username and/or password are invalid, the system should inform the user that they have entered an invalid username and/or password and prompt them to enter their credentials again.
- **SRS 4.2.3 REQ-4:** If the username and password are valid, the system should update the user type to be "encost-verified" and should provide the user with the ESGP Feature Options.
- **SRS 4.2.3 REQ-5:** Ten username and password pairs will be provided. The passwords should be encrypted before being stored in the application.

3.2.3 Test Type

Test Level: System-level black-box test
Test Technique(s): Equivalence Partitioning, Cause–Effect Graphing

- **Equivalence Partitioning** – to classify input types as valid, invalid, or empty.
- **Cause–Effect Graphing** – to map combinations of credential correctness to expected outcomes (e.g. success or failure).

Cause–Effect Logic Table

Rule ID	**Cause Conditions**	**System Effect**
R1	Valid username & valid password	Grant access, show ESGP options
R2	Invalid username *or* invalid password	Show error, re-prompt for credentials
R3	Empty username or password	Show error, re-prompt

Table 3.2: Cause–Effect Rule Table for SRS 4.2.3 – ESGP Account Login

3.2.4 Test Cases

Test ID	Input	Expected Output	Test Technique
TC07	Valid username and valid password	Login success; user becomes "encost-verified"; ESGP options shown	Valid input – Cause–Effect
TC08	Invalid username, valid password	Error message; re-prompt for input	Invalid username – Equivalence Partitioning
TC09	Valid username, invalid password	Error message; re-prompt for input	Invalid password – Equivalence Partitioning
TC10	Valid credentials in different case (e.g., "Admin", "PaSs123")	Success/failure depending on case sensitivity	Format edge case – Cause–Effect
TC11	Empty username or password	Error message; re-prompt	Boundary case – Equivalence Partitioning

Table 3.3: Black-box test cases for SRS 4.2.3 – ESGP Account Login using Equivalence Partitioning and Cause–Effect Graphing

Table 3.3 captures test cases that evaluate how the system responds to valid and invalid login attempts. These include username/password mismatches, empty fields, and casing variations. The tests ensure compliance with the authentication logic and secure handling of credentials.

3.3 Requirement #3

ESGP Feature Options Menu (Encost and Community Users)

3.3.1 Description

After the user is categorised (as Encost or Community) and authenticated if required, the ESGP system displays a feature selection menu. Verified Encost users are allowed to choose from three options: Upload Dataset, Visualise Graph, and View Summary Statistics. Community users, however, are limited to the visualisation feature only. The system must restrict access accordingly and initiate the appropriate function once a valid option is selected.

3.3.2 Functional Requirements Tested

- **SRS 4.3.3 REQ-1:** The ESGP Feature Options prompt should provide the user with a selection of features. Community users may only visualise the graph; Encost users may access all three features.
- **SRS 4.3.3 REQ-2:** Once a feature is selected, the corresponding feature prompt or process should begin.

3.3.3 Test Type

Test Level: System-level black-box test
Test Technique(s): Decision Table Testing

- **Decision Table Testing** – used to evaluate combinations of user types and selected options, ensuring that access control logic and system responses align with feature restrictions.

3.3.4 Decision Table: ESGP Feature Options

Test Case ID	**TC12**	**TC13**	**TC14**	**TC15**	**TC16**
User is Encost?	Yes	Yes	Yes	No	Any
Option = Upload (1)?	Yes	No	No	No	No
Option = Visualise (2)?	No	Yes	No	Yes	No
Option = Stats (3)?	No	No	Yes	No	No
Valid Option for User?	Yes	Yes	Yes	Yes	No
Action: Allow Access?	Yes	Yes	Yes	Yes	No
Action: Deny Access?	No	No	No	No	Yes
Action: Re-prompt?	No	No	No	No	No

Table 3.4: Decision Table for SRS 4.3.3 – ESGP Feature Options Access Rules

Table 3.4 outlines access behaviour depending on user type and the feature selected. Community users are restricted to graph visualisation, while Encost users have access to all three features. Invalid option selections are denied with appropriate feedback.

3.4 Requirement #4

Load Encost Smart Homes Dataset

3.4.1 Description

This requirement ensures that the ESGP system can correctly locate, access, and process the built-in Encost Smart Homes Dataset. It verifies whether the dataset exists, is readable, and conforms to the expected structure. This functionality is critical for enabling later visualisation and statistical features.

3.4.2 Functional Requirements Tested

- **SRS 4.4.3 REQ-1:** The Encost Smart Homes Dataset file should be located inside the system.
- **SRS 4.4.3 REQ-2:** The system should know the default location of the dataset.
- **SRS 4.4.3 REQ-3:** The system should be able to read the dataset line by line and extract the relevant device information.

3.4.3 Test Type

Test Level: System-level black-box test
Test Technique(s): Equivalence Partitioning, Boundary Value Analysis, All-Pairs Testing

- **Equivalence Partitioning** – for testing valid and invalid dataset conditions.
- **Boundary Value Analysis** – to test size-related edge cases like empty and large datasets.
- **All-Pairs Testing** – to efficiently cover critical combinations of dataset presence, readability, and structure.

3.4.4 All-Pairs Testing Explanation

All-Pairs Testing helps to reduce the total number of test cases by generating a minimal set of input combinations that covers all possible pairs of parameters. In this case, the ESGP system's response to the following three factors is evaluated:

- **Dataset Existence:** Exists / Missing

- **Readability:** Readable / Unreadable
- **Structure:** Valid / Malformed

3.4.5 Test Cases – All-Pairs Testing

Test ID	Dataset Existence	Readability	Structure	Expected Output
TC17	Exists	Readable	Valid	Dataset successfully loaded and parsed
TC18	Exists	Unreadable	Valid	Error message shown; file cannot be read
TC19	Exists	Readable	Malformed	Malformed lines skipped or logged; valid lines processed
TC20	Missing	Any	Any	Error message; dataset missing

Table 3.5: All-Pairs Test Matrix for SRS 4.4.3 – Load Encost Smart Homes Dataset

Supplementary Edge Case Tests

Test ID	Input Condition	Expected Output	Test Technique
TC21	Dataset has only header row (no data entries)	Warning shown, empty data structure returned	Boundary Value Analysis
TC22	Dataset contains 10,000+ entries	Dataset processed successfully without crash or memory issues	Boundary Value Analysis

Table 3.6: Supplementary test cases for edge scenarios (Boundary and Equivalence)

Table 3.5 presents All-Pairs Testing to validate how the system handles combinations of key dataset parameters. Table 3.6 includes additional edge cases that may not be captured in pairwise combinations but are important for robustness testing.

3.5 Requirement #5

Categorise Smart Home Devices

3.5.1 Description

Once a dataset is loaded, the ESGP system must identify and categorise each smart device according to its type (e.g., light, sensor, camera). This classification is essential for enabling both visualisation and statistical analysis. A device's category determines how it is displayed in the graph and counted in summary statistics.

3.5.2 Functional Requirements Tested

- **SRS 4.6.3 REQ-1:** The system should determine the device category for each device based on the dataset.
- **SRS 4.6.3 REQ-2:** The system should create a Device object for each entry, storing all associated information.

3.5.3 Test Type

Test Level: Component-level black-box test
Test Technique(s): Equivalence Partitioning, Error Guessing

- **Equivalence Partitioning** – to test across representative valid inputs from known device categories, assuming similar handling.
- **Error Guessing** – to test how the system handles invalid or incomplete device entries, such as unknown types or missing data.

This requirement is tested using a combination of **Equivalence Partitioning** and **Error Guessing**. The first ensures correct classification of valid types, while the latter identifies how gracefully the system responds to unexpected input. This combination improves robustness and reliability in data categorisation logic.

3.5.4 Test Cases

Test ID	Input	Expected Output	Test Technique
TC23	Device with type “light”	Categorised as “Lighting”	Equivalence Partitioning
TC24	Device with type “sensor”	Categorised as “Sensor”	Equivalence Partitioning
TC25	Device with type “camera”	Categorised as “Camera”	Equivalence Partitioning
TC26	Device with type “thermostat”	Categorised as “Climate”	Equivalence Partitioning
TC27	Device with unknown type “xyz”	Categorised as “Unknown” or logged as error	Error Guessing
TC28	Device with missing type field	Categorised as “Unknown” or handled gracefully	Error Guessing
TC29	Device with empty string as type	Error message or default category assigned	Error Guessing

Table 3.7: Black-box test cases for SRS 4.6.3 – Device categorisation using Equivalence Partitioning and Error Guessing

Table 3.7 summarises valid and invalid input categories to ensure that ESGP handles typical device classifications accurately and responds appropriately to anomalies or errors in the data.

3.6 Requirement #6

Build Graph Data Type (SRS 4.7)

3.6.1 Description

After loading a dataset and categorising the devices, the ESGP system must construct a graph data structure representing the smart home. Each device becomes a node, and connections between devices are represented as edges. If a new dataset is loaded, the graph must be cleared and rebuilt. A correctly built graph is essential for both the visualisation and statistics components of the system.

3.6.2 Functional Requirements Tested

- **SRS 4.7.3 REQ-1:** The system must create a graph data structure from the dataset.
- **SRS 4.7.3 REQ-2:** Each device in the dataset must become a node in the graph.
- **SRS 4.7.3 REQ-3:** The system must add edges to connect the nodes/devices if connections are defined in the dataset.
- **SRS 4.7.3 REQ-4:** If a new dataset is loaded, the graph must be cleared and rebuilt from scratch.

3.6.3 Test Type

Test Level: Component-level black-box test
Test Technique: Decision Table Testing

3.6.4 Decision Table: Graph Construction Conditions

Test ID	Dataset Loaded	Valid Devices	Connections Exist	New Dataset Loaded	Expected Outcome
TC30	Yes	Yes	Yes	No	Graph created with nodes and edges
TC31	Yes	Yes	No	No	Graph created with nodes only (no edges)
TC32	No	N/A	N/A	No	Error: Dataset not loaded
TC33	Yes	No	N/A	No	Error: Invalid device data
TC34	Yes	Yes	Yes	Yes	Previous graph cleared, new graph created

Table 3.8: Decision Table – Conditions for SRS 4.7 Build Graph Data Type

3.6.5 Decision Table: Graph Construction Actions

Test ID	Create Graph (A1)	Add Nodes (A2)	Add Edges (A3)	Update Graph (A4)	Show Error (A5)
TC30	Yes	Yes	Yes	No	No
TC31	Yes	Yes	No	No	No
TC32	No	No	No	No	Yes
TC33	No	No	No	No	Yes
TC34	Yes	Yes	Yes	Yes	No

Table 3.9: Decision Table – Actions for SRS 4.7 Build Graph Data Type

Tables 3.8 and 3.9 define the expected system behaviour based on various input combinations. These tables confirm that the ESGP system only constructs a graph under appropriate conditions and handles errors or reinitialisation correctly and consistently.

3.7 Requirement #7

Graph Visualisation (SRS 4.8)

3.7.1 Description

The ESGP must allow users to visualise the graph constructed from the loaded dataset, where devices are represented as nodes and their connections as edges. Visualisation should be accurate, user-accessible, and only available once a valid graph exists. If the graph is missing or incomplete, the system must provide a helpful error message.

3.7.2 Functional Requirements Tested

- **SRS 4.8.3 REQ-1:** The ESGP must allow the user to visualise the graph.
- **SRS 4.8.3 REQ-2:** The visualisation should reflect the current graph structure accurately.
- **SRS 4.8.3 REQ-3:** If the graph is not available or invalid, the system should display an appropriate message.

3.7.3 Test Type

Test Level: System-level black-box test
Test Technique(s): Error Guessing, Boundary Value Analysis

- **Error Guessing** – to simulate invalid, missing, or incomplete graph data.
- **Boundary Value Analysis** – to validate performance and output for graphs with minimum or large-scale node/edge combinations.

3.7.4 Test Cases

Test ID	Input Condition	Expected Output	Test Technique
TC35	Graph with 1 node and no edges	Single node visualised; no connections drawn	Boundary Value Analysis
TC36	Graph with 1000+ nodes and edges	Full graph visualised successfully without lag or crash	Boundary Value Analysis
TC37	No dataset loaded before visualisation attempt	Error message: Graph not available	Error Guessing
TC38	Dataset loaded but graph build failed	Error message: Graph invalid or incomplete	Error Guessing
TC39	Graph has only disconnected nodes	Nodes visualised without edges	Boundary / Error Guessing

Table 3.10: Black-box test cases for SRS 4.8 – Graph Visualisation

Table 3.10 lists key test cases that validate how the ESGP handles visualisation under various graph states. These include both functional boundaries (e.g., size extremes) and failure modes (e.g., unbuilt graphs), ensuring the system remains robust and informative in all cases.

4 White-box testing

4.1 <Summary Statistic: Devices Per Category Within Each Region> Pseudocode

4.1.1 Description

This pseudocode corresponds to SRS requirement **4.9.3 REQ-3**: *The system should return the number of devices per category within each region.* The function processes the

graph and returns grouped statistics by region and category.

```
void computeCategoryStatsPerRegion(Graph graph)
   INITIALIZE stats as empty Dictionary

   // Branch 1
   IF graph IS NULL OR graph.nodes IS EMPTY THEN
          PRINT "Error:  Graph not available"
          RETURN stats
   ENDIF

   FOR EACH node IN graph.nodes DO
          // Branch 2
          IF node.region IS NULL OR node.category IS NULL THEN
          PRINT "Warning:  Node missing region or category"
          CONTINUE
          ENDIF

          // Branch 3
          IF stats DOES NOT CONTAIN node.region THEN
          INITIALIZE stats[node.region] AS empty Dictionary
          ENDIF

          // Branch 4
          IF stats[node.region] DOES NOT CONTAIN node.category THEN
          SET stats[node.region][node.category] = 0
          ENDIF

          INCREMENT stats[node.region][node.category]
   END FOR

   RETURN stats
```

4.2 Branch Coverage Testing

4.2.1 Identified Branches

- **Branch 1:** Graph is null or empty.
- **Branch 2:** Node has null region or category.
- **Branch 3:** Region not in stats dictionary.
- **Branch 4:** Category not in stats[region] dictionary.

4.2.2 Test Cases

Test ID	Input Description	Expected Output	Branches Covered
TC36	Graph is `null` or has no nodes	Print error and return empty stats	Branch 1
TC37	Graph with one node missing region/category	Node skipped, warning printed	Branch 2
TC38	Graph with node [A, R1]	`stats = {R1:{A:1}}`	Branches 3 and 4
TC39	Graph with [R1:A], [R1:B]	`stats = {R1:{A:1,B:1}}`	Branch 4 reused region
TC40	Graph with [R1:A], [R2:A]	`stats = {R1:{A:1},R2:{A:1}}`	Branch 3 reused category

Table 4.1: White-box test cases for summary statistics generation (per category per region)

4.2.3 Branch Coverage Summary

- **Branch 1** – Covered by TC36
- **Branch 2** – Covered by TC37
- **Branch 3** – Covered by TC38 and TC40
- **Branch 4** – Covered by TC38 and TC39

Conclusion: The test suite above achieves full (100%) branch coverage by validating all possible decision paths through the pseudocode.

5 Mutation Testing

5.1 Mutant #1 – Logical Operator Mutation

Original:

```
IF graph IS NULL OR graph.nodes IS EMPTY THEN
```

Mutated:

```
IF graph IS NULL AND graph.nodes IS EMPTY THEN
```

Effect: This mutant changes the logical condition to a more restrictive check. It may fail to detect situations where the graph is null but `graph.nodes` is not empty, or vice versa.

5.2 Mutant #2 – Skipped Initialization of Dictionary

Mutated:

```
IF region not in summaryStats THEN
    // Do not initialize dictionary
ENDIF
```

Effect: If a region is not already in `summaryStats`, the system will attempt to increment a non-existent entry, resulting in a runtime error or crash.

5.3 Mutant #3 – Reverse Operation (Decrement Instead of Increment)

Mutated:

```
INCREMENT summaryStats[region][category] by -1
```

Effect: This mutation causes the statistics to be incorrect, showing negative device counts rather than accumulating them.

5.4 Mutant #4 – Negated Region Check

Mutated:

```
IF region in summaryStats THEN
```

Effect: The logic is flipped. The system skips initialization for new regions and may incorrectly overwrite or access existing entries.

5.5 Mutation Score Evaluation

5.5.1 Test Set 1

Input	**Expected Output**
category: 'A', region: 'R1' category: 'B', region: 'R1' category: 'A', region: 'R2'	`R1: {A:1, B:1}, R2: {A:1}`

Table 5.1: Mutation Testing – Test Set 1 (Input and Expected Output)

Mutant 1	**Mutant 2**	**Mutant 3**	**Mutant 4**
Same as expected (Not Detected)	Runtime error due to uninitialized dictionary (Detected)	Negative counts returned (Detected)	Initialization skipped (Detected)

Table 5.2: Mutation Testing – Test Set 1 (Mutant Detection Results)

Mutation Score (Set 1): Total Mutants Applicable: 4
Mutants Detected: 3
Mutation Score = (3 / 4) × 100 = **75%**

5.5.2 Test Set 2

Input	**Expected Output**
`null` graph	Error message: graph not available

Table 5.3: Mutation Testing – Test Set 2 (Input and Expected Output)

Mutant 1 Output	**Mutant 2**	**Mutant 3**	**Mutant 4**
No output (Detected)	N/A	N/A	N/A

Table 5.4: Mutation Testing – Test Set 2 (Mutant Detection Results)

Mutation Score (Set 2): Total Mutants Applicable: 1
Mutants Detected: 1
Mutation Score = (1 / 1) × 100 = **100%**